JavaScript Notes: var, let, and const

When declaring variables in JavaScript, you can use var, let, or const. Although they look similar,

they behave differently — especially in scope, redeclaration, and mutability.

1. var — Function Scoped

• Function-scoped (not block-scoped).

• Can be redeclared and updated.

• Hoisted and initialized as undefined.

Example:

```
function testVar() {
  if (true) {
    var x = 10;
  }
  console.log(x); // ✅ Works! Output: 10
}
testVar();
```

Even though x was declared inside the i block, it's still accessible outside because var only respects unction scope, not block scope.

Redeclaration example:

```
var name = "Alice";
var name = "Bob"; // ✅ Allowed
console.log(name); // Output: Bob
```

Hoisting example:

```
console.log(a); // Output: undefined (hoisted but not
assigned)
var a = 5;
```

2. let — Block Scoped

- Block-scoped (works only inside {}).
- Can be updated but not redeclared.
- Hoisted but not initialized (Temporal Dead Zone).

Example:

```
function testLet() {
  if (true) {
    let y = 20;
    console.log(y); // ✅ Works: 20
  }
  console.log(y); // ❌ Error: y is not defined
}
testLet();
```

Updating value:

```
let score = 10;
score = 15; // ✅ Allowed
```

Redeclaration not allowed:

```
let city = "Singapore";
let city = "Tokyo"; // ❌ SyntaxError
```

Hoisting behavior:

```
console.log(num); // ❌ ReferenceError
let num = 100;
```

let is hoisted but not accessible before its declaration — it stays in the Temporal Dead Zone until the declaration line is reached.

3. const — Block Scoped and Constant

- Block-scoped like let.
- Must be initialized immediately.
- Cannot be updated or redeclared.
- Object contents can still be modified.

Example:

```
const PI = 3.14;
console.log(PI); // 3.14

PI = 3.14159; // ❌ TypeError: Assignment to constant
variable
```

With objects:

```
const person = { name: "Alice", age: 20 };

// Allowed: modifying the property
person.age = 21; // ✅ Works

// Not allowed: reassigning the variable
person = { name: "Bob", age: 22 }; // ❌ Error
```

4. Summary Table

Feature	var	let	const
Scope	Function	Block	Block
Hoisted	Yes (initialized as undefined)	Yes (TDZ applies)	Yes (TDZ applies)
Redeclaration	✅ Allowed	❌ Not allowed	❌ Not allowed
Reassignment	✅ Allowed	✅ Allowed	❌ Not allowed
Must initialize	❌ No	❌ No	✅ Yes
Common use	Older code / function scope	Modern variable declarations	Constants and references

5. Practical Example: Comparing All Three

```
function demo() {
  if (true) {
    var a = 1;
    let b = 2;
    const c = 3;
  }

  console.log(a); // ✅ Works (var is function-scoped)
  console.log(b); // ❌ Error (block-scoped)
  console.log(c); // ❌ Error (block-scoped)
}
demo();
```

Best Practices:

1. Avoid using var in modern JavaScript.

2. Use let for variables that change.

3. Use const for constants or references.